

Blindlaunch: System Architecture

Design document for a sealed-bid token launch on Solana. This is a draft. Details about Arcium and Meteora come from their public docs and need a final check against the current versions before you build.

1. Overview

Blindlaunch sells a new token through sealed bids. Everyone bids in secret, one price is computed for all, and trading opens at that price. Because order does not matter and nobody can see bids, speed earns nothing and bots lose their edge.

Three design principles keep the build small and safe:

1. **Money stays in the Solana program.** Deposits, token transfers and refunds are ordinary, public, auditable program logic.
2. **Arcium only computes.** The private part is one calculation: the clearing price and each bidder's share.
3. **Build the safety net first.** A working auction with visible bids comes before the private version, so there is always something to demo.

2. Components

Component	Where it runs	Job	Technology
Web app	Browser	Connects the wallet, encrypts bids on the bidder's device, shows auction status and results	React or Next.js, Solana wallet adapter, <code>@arcium-hq/client</code>
Solana program	On-chain	Holds the rules, deposits and state, and pays everyone out	Anchor, built with the Arcium CLI
Arcium circuit	Arcium network	Computes the clearing price and each bidder's share from encrypted bids	Arcis (Rust)

Component	Where it runs	Job	Technology
Keeper script	Off-chain	Calls close and settle once the deadline has passed. It needs no special rights, since anyone can call those actions	TypeScript script
Meteora pool	On-chain	Trading venue that opens at the clearing price	Meteora DAMM v2, <code>createCustomPool1</code> in the TypeScript SDK

3. On-chain data (proposed)

These are the accounts the Solana program stores. Field names are a starting point.

Auction account

Field	Meaning
authority	The team that created the auction. Cannot bid
token mint, quote mint	The token for sale and the currency it is paid in (USDC)
supply	Number of tokens for sale
min price	Lowest price the team accepts
price grid	Number of allowed prices (for example 64) and the step between them
max bidders	Fixed bid slots (for example 32). Unused slots are padded with zero-bids
deposit per bidder	The same fixed deposit for everyone
deadline, timeout	When bidding closes, and when refunds open if no result arrives
status	Open, Computing, Cleared, Settled or Cancelled

Field	Meaning
clearing price, total sold	Filled in after the computation
token vault, USDC vault	Where the tokens and the deposits are held

Bid account

Field	Meaning
auction, bidder	Which auction, and which wallet
deposit	The fixed deposit locked by this bidder
encrypted bid	Price step and amount, encrypted, plus the nonce needed to decrypt
slot index	Position in the fixed-size bid list
settled flag	Whether tokens and refund have been paid out

4. Lifecycle

1. **Create.** The team locks the tokens in the token vault and sets the rules. The auction status is Open.
2. **Bid.** A bidder encrypts their price and amount in the browser, then sends the encrypted bid with the fixed deposit. The chain shows only scrambled data and a deposit of the standard size.
3. **Close.** After the deadline, anyone (usually the keeper) calls close. The program queues the price calculation on Arcium and sets the status to Computing.
4. **Compute.** The Arcium network runs the circuit on the encrypted bids and returns the clearing price, the total sold and each winner's share to the program.
5. **Settle.** The program sends tokens to winners at the single price, refunds each bidder's unspent deposit, pays the team, and opens the Meteora pool at the clearing price.
6. **Refund path.** If no result arrives before the timeout, the auction is Cancelled. Every bidder can reclaim their deposit and the team can reclaim the tokens.

5. The circuit

Inputs: a fixed list of padded bids (price step and amount per slot), the supply, and the price grid.

Outputs: the clearing price, the total sold, and each bidder's allocation.

Method:

1. For a candidate price, demand is the sum of the amounts from every bid at or above that price.
2. Binary search over the price grid finds the highest price where demand still covers the supply. This takes only a handful of rounds, and each round is a simple scan.
3. Bids above the clearing price are filled in full. Bids exactly at the clearing price share what is left in proportion to their size. Compute the ratio once, then multiply, because division is the expensive operation.
4. Round in the protocol's favour and send any leftover dust to the team, so the total can never exceed the supply.

Why not sort the bids: sorting inside Arcium costs roughly $n \times \log^2 n$ times the bit size, while the binary search above is far cheaper.

Arcis rules the circuit must follow:

- Loops have a fixed count and no `while`, `break` or `continue`.
- Lists have a fixed size, so use arrays padded with zero-bids, never `Vec`.
- Both sides of an `if` always run, so cost is the sum of both branches.
- Use whole numbers for prices and amounts, since decimals are emulated and approximate.

What becomes public: the clearing price and total sold. Winners' allocations are visible once tokens are transferred to them, because token transfers are public on Solana. Losing bids are never revealed.

6. Privacy and security

Risk	How it is handled
A large deposit reveals a large bid	Every bidder locks the same fixed deposit and gets the unused part refunded

Risk	How it is handled
Bots snipe the pool right after launch	Meteora's fee scheduler charges very high fees at the start and lowers them over time. The DAMM v2 default is reported as 50% falling to 0.25% over 120 minutes, to be confirmed
The team bids on its own sale	The authority cannot bid, and a minimum price and bid caps apply
Arcium computation stalls	Timeout, then refund path for bidders and the team
Trust in the Arcium nodes	Privacy is only as strong as Arcium's security assumptions. State them accurately in the pitch
Bid shading	A known property of uniform-price auctions. Disclose it as a limitation
Splitting a large bid across wallets	A known limit of the fixed-deposit design. Disclose it as a limitation
Program bugs	Keep the program small, test every action and the refund path

7. Build order

1. Write the price rule in ordinary code and test it, including the four-bidder example that gives \$0.60.
2. Build the Solana program with visible bids and the refund path. This is the safety net.
3. Move the price rule into the Arcis circuit with small sizes (4 bidders, 8 price steps), then scale up.
4. Add bid encryption in the web app.
5. Add the Meteora pool step. If time is short, stop at settle and describe it.
6. Build the demo: the same launch run as an open launch (a bot wins) and as Blindlaunch (no bot advantage).

8. Check before building

- The current Arcium version and its circuit size limits. The examples target an older release than the latest toolchain.
- Whether the circuit can return each winner's allocation to the program in the form settlement needs.
- Whether your target network (devnet or mainnet) is supported for the Arcium cluster you plan to use.
- Whether Docker is required for local testing, and which operating system you will use. Arcium does not support Windows natively.
- Which Meteora pool type supports the anti-sniper fee scheduler you want, and how Blindlaunch differs from Meteora's Alpha Vault.